Module 01

Application Layer

Principles of Network Applications

1. Introduction

- A network application starts as an idea that later becomes a real working system.
- The goal may be to help users, provide a service, generate income, or simply be interesting to build.
- To turn an idea into a real application, developers must understand how network applications work.

---

2. Core Concept of Network Application Development

- At the heart of any network application is a set of **programs running on different end systems**.
- These programs **communicate with each other over the network**.
- End systems include: desktops, laptops, tablets, smartphones, etc.

---

3. Example: Web Application Structure

A typical web application has two main programs:

1. **Browser program**
    - Runs on the user's device.
2. **Web server program**
    - Runs on the server host (web server).

Both programs exchange messages through the Internet.

---

4. Example: P2P File Sharing

- In a peer-to-peer (P2P) system, **each host runs the same or similar program**.
- Every device in the network participates in sharing files.
- There is **no central server**, and all peers communicate directly.

---

5. Software Requirements for New Network Applications

- When building a new network application, you must write software that runs on **multiple end systems**.
- Programming languages typically used: **C, Java, Python**, etc.
- **You do NOT write software for network-core devices** like routers or link-layer switches.

---

6. Why We Don't Program Network-Core Devices

- Core devices do not support application-layer programs.
- They only operate at lower layers (network layer and below).
- Even if you want to run your application on routers/switches, you cannot.
- This keeps the network simple and fast.

---

7. Key Design Principle

- Application software is always **confined to end systems**, not the core of the network.
- This design makes it easy to:
    - Develop network applications quickly
    - Deploy them widely
    - Support millions of different apps on the Internet

Network Application Architectures

1. What is Network Application Architecture?

- Before writing code for a network application, developers must choose an **application architecture**.
- Application architecture defines **how the application is structured** across different end systems.
- It is different from the **network architecture** (like the 5-layer Internet model).
- Most modern network applications use one of two major architectures:
    1. **Client–Server Architecture**

2. **Peer-to-Peer (P2P) Architecture**

---

2. Client–Server Architecture

A well-known and widely used architecture.

Key Features

1. **Always-on Server**
    - A dedicated server is always running to handle requests from clients.
    - Example: Web server for browsers.
2. **Clients Communicate With Server, Not With Each Other**
    - Two clients do not directly communicate.
    - All communication goes through the server.
3. **Fixed, Well-Known Server Address**
    - Server has an IP address known to all clients.
    - Clients send requests to this address.
4. **Server responds to client requests**
    - Example: Browser requests an object (image, page, file); server sends it back.

Examples of Client–Server Applications

- Web (HTTP)
- FTP
- Telnet / SSH
- Email systems

Scalability Issue

- If there is only one server handling all client requests, it may become overloaded.
- Large companies use a **data center** (many servers working together) to handle millions of requests.
- Example: Google uses 30–50 data centers around the world.

---

3. Peer-to-Peer (P2P) Architecture

Main Characteristics

1. **No Always-on Server**
    - No fixed server; all participating devices help each other.
2. **Peers Act as Both Client and Server**
    - A peer can request files and also upload files to other peers.
    - Devices are owned by users (laptops, desktops), not a company.
3. **Direct Communication Between Peers**
    - Data is exchanged directly without going through a central server.
4. **Highly Scalable (Self-scalability)**
    - Each peer adding to the system also adds service capacity.
    - As the number of users increases, the system becomes stronger.

Examples of P2P Applications

- File sharing (BitTorrent)
- Skype (earlier versions)
- Internet telephony
- P2P streaming platforms

---

4. Advantages of P2P Architecture

1. Self-Scalability

- When peers join, they contribute bandwidth, storage, and processing.
- More peers → More resources → Better performance.

2. Cost-Effective

- No dedicated servers needed.
- Service providers don’t need to maintain large data centers.

---

5. Challenges in P2P Architecture

Even though P2P is powerful, it faces some difficulties:

1. ISP Friendliness

- ISPs provide more downstream bandwidth than upstream.
- P2P applications often require high **upstream** bandwidth.
- This creates stress on ISP networks.

2. Security Issues

- Highly distributed nature makes it harder to control.
- More exposure to attacks (malware, unauthorized access).

3. Incentives

- P2P depends on users contributing resources.
- Challenge: How to encourage users to share bandwidth/storage.

Feature	**Client–Server**	**P2P**
Server	Always-on server	No dedicated server
Ownership	Server owned by provider	Peers owned by users
Communication	Client → Server	Peer ↔ Peer
Scalability	Limited, depends on server	Highly scalable
Examples	Web, FTP, Email	BitTorrent, Skype, P2P sharing
Cost	High (server maintenance)	Low (no central server)
Challenges	Server overload	ISP load, security, incentives

Processes Communicating

1. What is a Process?

- In operating systems, a **process** is a program running inside an end system.
- Network applications consist of **processes running on different hosts** that need to communicate.
- Communication between processes on the **same host** uses OS interprocess communication.
- In networking, we focus on **processes on different hosts** communicating through the computer network.

---

2. How Processes Communicate

- Two processes communicate by **exchanging messages**.
- A **sending process** creates and sends messages into the network.
- A **receiving process** reads these messages and may send response messages back.
- All such communication happens at the **application layer** of the Internet protocol stack.

Client and Server Processes

1. Concept

- Network applications consist of **pairs of processes** that exchange messages.
- Example:
    - Web: browser process ↔ web server process
    - P2P file sharing: peer A ↔ peer B

---

2. Client vs. Server

We label processes based on **who initiates communication**:

Client Process

- The process that **initiates** communication.
- Starts the session.

Server Process

- The process that **waits to be contacted**.
- Responds to client requests.

Examples:

- Web:
    - Browser = client
    - Web server = server
- P2P:
    - If Peer A asks Peer B for a file → **Peer A is client**, **Peer B is server**
    - In another session, roles may reverse.

---

3. Interface Between Processes and the Network

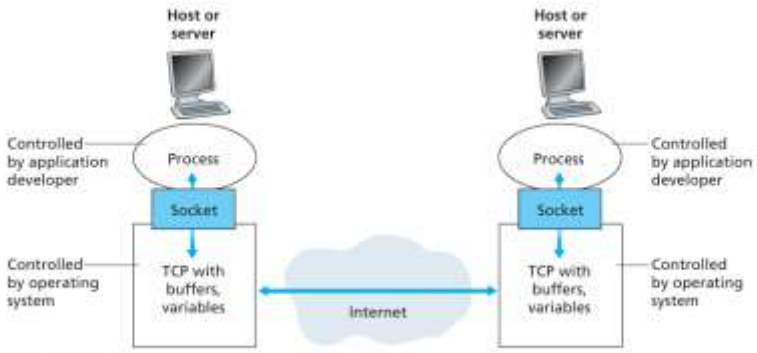

Figure 2.3 • Application processes, sockets, and underlying transport protocol

1. Socket

- A **socket** is the interface between the application process and the computer network.
- It is like a "**door**" through which a process sends and receives messages.
- Two controls:
    - Application controls the **application-layer side** of the socket.
    - Operating system controls the **transport-layer side** (TCP/UDP, buffers, variables).

---

2. How Communication Happens

- To send a message:
    - Process passes the message to the socket.
    - The OS and transport layer deliver it across the network.
- To receive a message:
    - The socket at the destination delivers the message to the receiving process.

---

3. Application Programming Interface (API)

- The socket serves as the **API** for communication.
- Developer can:
    1. Choose the transport protocol (TCP/UDP).
    2. Set some parameters (buffer sizes, segment size limits).

---

4. Addressing Processes

To send data to a process on another host, two things are needed:

1. IP Address

- Identifies the **host**.
- Unique to each device on the Internet.

2. Port Number

- Identifies the **specific process** (application) on the host.
- Example port numbers:
    - HTTP: 80
    - HTTPS: 443
    - SMTP (mail server): 25

Together:
(IP address + port number) uniquely identifies a receiving process.

Transport Services Available to Applications

A **transport-layer protocol** provides communication services between application processes running on different hosts. When developing an application, developers must choose a **transport protocol** (e.g., TCP or UDP) based on the services they need.

Transport services are classified into **four main dimensions**:

1. **Reliable Data Transfer**
2. **Throughput**
3. **Timing**
4. **Security**

---

1. Reliable Data Transfer

What is it?

- In computer networks, data can be **lost**, **corrupted**, or **discarded** due to router overload, congestion, or bit errors.
- Some applications cannot tolerate any data loss.

When required?

- Critical applications such as:
    - File transfer
    - Web document transfer
    - Email
    - Financial transactions
- In these applications, every bit must be delivered correctly.

How it works?

- If the transport protocol ensures all data is delivered **completely and correctly**, it provides **process-to-process reliable data transfer**.
- Sender can pass data to the socket with confidence it will arrive error-free.

Loss-Tolerant Applications

- Some applications can tolerate occasional data loss:
    - Audio/video streaming
    - Internet telephony
    - Online video services
- These applications accept small glitches instead of perfect reliability.

---

2. Throughput

Definition

- The rate at which the sending process can deliver bits to the receiving process.
- Available throughput may vary due to other active network sessions.

Guaranteed Throughput

- Some applications need a **minimum guaranteed data rate**.
- Example:
  Internet telephony encoding voice at **32 kbps** requires at least that much throughput.

Bandwidth-Sensitive Applications

- Applications requiring a minimum throughput:
    - Real-time voice
    - Streaming
    - Certain multimedia applications

Elastic Applications

- Applications that can adjust to any available throughput:
    - Email
    - File transfer
    - Web browsing
- They use as much bandwidth as available.

---

3. Timing

Definition

- Timing requirements specify **how quickly** data must be delivered.

Examples

- An application may require that each message arrive **within 100 milliseconds**.
- Such guarantees are essential for:
    - Internet telephony
    - Video conferencing
    - Online gaming
    - Virtual reality

Impact of Delay

- High delay causes:
    - Breaks in conversation (VoIP)
    - Lag in games
    - Poor user experience in real-time interaction

Non Real-Time Applications

- For normal applications like email or file download:
  Low delay is preferred, but **strict timing is not required**.

---

4. Security

Transport layer can provide:

1. Confidentiality

- Encrypting data at the sender
- Decrypting at the receiver
- Prevents eavesdroppers from reading data during transmission

2. Data Integrity

- Ensures data is not modified during transfer

3. End-point Authentication

- Confirms the identity of communicating parties

Transport-layer security is commonly provided by **TLS (Transport Layer Security)**.

Service	Meaning	Example Applications
Reliable Data Transfer	Ensures no data loss	File transfer, email, banking
Throughput	Minimum data rate required	Voice call, video streaming
Timing	Data must arrive within time limit	Gaming, teleconferencing
Security	Encryption, integrity, authentication	Secure web browsing (HTTPS)

Transport Services Provided by the Internet

The Internet primarily provides **two transport-layer protocols**:

1. **TCP (Transmission Control Protocol)**
2. **UDP (User Datagram Protocol)**

Application developers must choose either TCP or UDP based on the service requirements of their applications.

---

1. Requirements of Network Applications

Different applications need different levels of service.
Figure 2.4 in the textbook lists these requirements:

Examples

Application	Data Loss	Throughput	Time-Sensitive
File Transfer	No loss	Elastic	No

E-mail	No loss	Elastic	No
Web documents	No loss	Elastic (few kbps)	No
Internet telephony / Video conferencing	Loss-tolerant	Audio: few kbps–1 Mbps Video: 10 kbps–5 Mbps	Yes (100s of ms)
Streaming stored audio/video	Loss-tolerant	Same as above	Yes (few seconds)
Interactive games	Loss-tolerant	Few kbps–10 kbps	Yes (100s of ms)
Instant messaging	No loss	Elastic	Sometimes

---

2. TCP Services

TCP provides **two major services**:

(1) Connection-Oriented Service

- Before sending data, the client and server **exchange control information**.
- This is known as **TCP handshaking**.
- After the handshake, a **TCP connection** is established.
- Full-duplex communication: both sides can send data simultaneously.
- When communication ends, TCP **tears down** the connection.

(2) Reliable Data Transfer

- TCP guarantees all bytes are:
    - Delivered **without error**
    - Delivered **in order**
    - **Without duplicates**
- Suitable for:
    - File transfer (FTP)
    - Web browsing (HTTP)
    - Email (SMTP)
    - Remote login (Telnet)

TCP Congestion Control

- Reduces sending rate when network congestion occurs.
- Protects the **overall health of the Internet**.
- Ensures fair bandwidth sharing among TCP flows.

---

3. Security Enhancement: SSL (Secure Sockets Layer)

TCP by itself does **not** provide encryption.

SSL is built **on top of TCP** and provides:

- **Confidentiality** (encryption)
- **Integrity**
- **End-point authentication**

Applications using SSL follow this flow:

1. Application sends **cleartext** to SSL.
2. SSL encrypts and sends data to TCP.
3. At the receiving host, SSL decrypts the data.
4. Application receives cleartext.

SSL behaves like a third transport protocol but is implemented at the **application layer**.

---

4. UDP Services

UDP is a **lightweight, minimal** transport protocol.
UDP provides:

(1) Unreliable Data Transfer

- No guarantee that:
    - The message will arrive
    - The message will arrive **in order**
    - The message will arrive **only once**

(2) No Connection Establishment

- UDP is **connectionless**.
- No handshaking before data transfer.
- Lower delay than TCP.

(3) No Congestion Control

- UDP can send data at any rate.
- Suitable for applications that need:
    - Low delay
    - Tolerance for loss
    - High-speed streaming

Common UDP Uses

- Internet telephony (VoIP)
- Live video streaming
- Online gaming
- Real-time applications

---

5. Services NOT Provided by the Internet Transport Layer

Even though applications need many services, **TCP and UDP do NOT provide**:

Guaranteed Throughput

- No guaranteed minimum rate.
- Throughput fluctuates depending on congestion and competing traffic.

Timing Guarantees

- No strict control over delay.
- Applications like online gaming or video calls must tolerate variable delays.

Native Security (by TCP/UDP)

- Only SSL/TLS (above TCP) provides security.

Despite these limitations, many time-sensitive applications still work well due to clever design and adaptive techniques.

---

6. Transport Protocols Used by Popular Applications (Figure 2.5)

Application	Application Layer Protocol	Underlying Transport
Email	SMTP	TCP
Remote terminal	Telnet	TCP

Web	HTTP	TCP
File transfer	FTP	TCP
Streaming multimedia	HTTP (YouTube)	TCP
Internet telephony	SIP, RTP, or proprietary	UDP or TCP

Key Insight:

- Most applications use **TCP** because they need reliable delivery.
- **Internet telephony** often uses **UDP** because it tolerates loss but needs low delay.

Application-Layer Protocols

Network processes communicate by sending messages through sockets.
But to understand communication **structure**, **format**, and **rules**, we use **application-layer protocols**.

---

1. What is an Application-Layer Protocol?

An **application-layer protocol** defines **how the processes of a network application communicate** with each other, even when running on different hosts.

It specifies:

(1) Types of Messages

- Request messages
- Response messages
- Control or data messages

(2) Message Syntax

- The format of messages
- The fields inside messages
- How fields are ordered and separated

(3) Semantics of Message Fields

- The meaning of each field
- What each field indicates or instructs

(4) Rules of Communication

- When processes send messages

- How they respond
- Conditions and timing for message exchange

These rules ensure that two processes can **understand** each other even when on different systems.

---

2. Public vs Proprietary Protocols

Public Protocols

- Documented in **RFCs (Request for Comments)**.
- Openly available for anyone to implement.
- Example:
    - **HTTP** (HyperText Transfer Protocol) – RFC 2616
      Any browser that follows HTTP RFC rules can communicate with any web server following the same rules.

Proprietary Protocols

- Not publicly available.
- Used privately by companies.
- Example:
    - **Skype** uses proprietary application-layer protocols for communication.

---

3. Application-Layer Protocol vs Network Application

It is important to understand:

A network application ≠ application-layer protocol

The **application-layer protocol** is only *one part* of the overall application.

Example 1: Web Application

A web application has many components:

- HTML document format
- Web browsers (Chrome, Firefox, IE)
- Web servers (Apache, Microsoft IIS)
- **HTTP protocol** (defines message format between browser and server)

HTTP is only the protocol piece, even though it is crucial.

Example 2: E-mail Application

Components include:

- Mail servers
- Mail clients (Outlook, Thunderbird)
- Message format standards
- Protocols for sending messages:
    - **SMTP** (Simple Mail Transfer Protocol)
- Rules for message delivery between mail servers

Again, SMTP is only **one** part of the entire e-mail application.

---

4. Importance of Application-Layer Protocols

They ensure:

- Interoperability between different software applications
- Standardized communication
- Smooth functioning of the web, email, file transfer, and other services

Each protocol guarantees that applications built by different developers can communicate universally.

---

5. Examples of Application-Layer Protocols

Application	**Protocol**
Web browsing	HTTP
E-mail sending	SMTP
E-mail access	POP3, IMAP
File transfer	FTP
Internet telephony	SIP, RTP
Remote terminal access	Telnet

The Web and HTTP

1. Introduction to the Web

Before the early 1990s, the Internet was mainly used by:

- Researchers
- Academics
- University students

Uses included:

- Remote login
- Transferring files
- Reading news
- Sending/receiving email

The Internet was mostly unknown to the general public.

The Web Revolution (Early 1990s)

- The **World Wide Web (WWW)** was introduced.
- It was the first Internet application to capture global attention.
- It dramatically:
    - Changed how people interact
    - Connected the world
    - Made the Internet the dominant data network

Why the Web Became Popular

1. **On-demand access**
   Users access content when *they* want it, unlike radio/TV which broadcast at fixed times.
2. **Interactivity**
   Web pages allow hyperlinks, form submissions, graphics, JavaScript, etc.
3. **Everyone can publish**
   Very easy for individuals to publish information online.
4. **Massive ecosystem**
   After 2003, apps like YouTube, Gmail, and Facebook made the Web even more powerful.

2.2.1 Overview of HTTP

What is HTTP?

- HTTP = **HyperText Transfer Protocol**
- Application-layer protocol of the Web
- Defined in RFC 1945 and RFC 2616
- Implements communication between:
    - **Client program** (browser)
    - **Server program** (web server)

HTTP governs:

- Message types
- Message structure
- How clients and servers exchange web objects

HTTP Fundamentals

Web Page

A **Web page** (document) consists of:

- A **base HTML file**
- Multiple objects (images, videos, scripts, CSS files, applets, etc.)

An **object** is any file addressable by a **URL**.

Example

If a webpage contains:

- 1 HTML file
- 5 JPEG images
  Then total objects = **6**.

URL Structure

A URL has:

1. **Hostname** (e.g., www.school.edu)
2. **Path name** (e.g., /images/picture.gif)

Example:

http://www.someSchool.edu/someDepartment/picture.gif

Client and Server in HTTP

Client

- The Web browser (Chrome, Firefox, Edge)
- Sends HTTP requests
- Receives HTTP responses

Server

- Web server software (Apache, Nginx, Microsoft IIS)
- Hosts web objects
- Sends requested pages to clients

Browser = Client
Web server = Server

How HTTP Works (Request–Response Cycle)

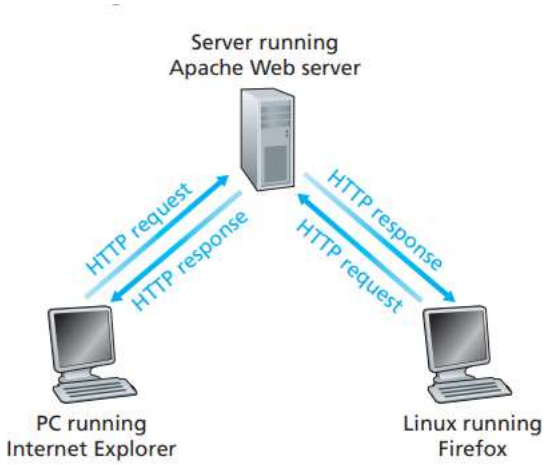

Figure 2.6 ♦ HTTP request-response behavior

1. **Browser clicks a hyperlink → sends HTTP request**
2. **Server receives request → sends HTTP response containing objects**
3. **Response may contain HTML text, images, scripts, etc.**

Use of TCP

HTTP **uses TCP**, not UDP.

Steps:

1. Client initiates a **TCP connection** to the server.
2. Once TCP connection is established:
    - Browser sends HTTP request through client socket.
    - Server reads request from socket.
    - Server sends response through socket.
3. TCP guarantees:
    - No data loss
    - No reordering
    - Fully reliable delivery

HTTP relies on TCP to ensure complete, error-free transmission.

Stateless Nature of HTTP

HTTP is a **stateless protocol**.

This means:

- The server does NOT store information about the client.
- If the same client requests the same object twice, the server treats it as two separate requests.
- Server does not remember:
    - Past actions
    - User identity
    - Session details (unless cookies or other mechanisms are added)

Statelessness simplifies server design and makes the Web scalable.

Non-Persistent and Persistent Connections

In many Internet applications, a client and server communicate for some time, exchanging multiple requests and responses.
When using TCP, the application developer must choose:

1. **Non-persistent connection**
   → one TCP connection per request–response pair

2. **Persistent connection**
   → one TCP connection used for multiple requests–responses

HTTP supports both, but **modern browsers use persistent connections by default**.

---

1. Non-Persistent Connections

Definition

For **each object requested**, a **separate TCP connection** is created, used, and closed.

Steps (Example: Web page with 1 HTML + 10 JPEGs)

Let the base URL be:

http://www.someSchool.edu/someDepartment/home.index

What happens:

1. Client initiates TCP connection to server at port 80.
2. Client sends HTTP request (GET) for /someDepartment/home.index.
3. Server retrieves the HTML file, encapsulates it in an HTTP response, and sends it.
4. Server closes the TCP connection. (Client closes after receiving full response.)
5. Client receives the HTML, parses references to 10 images.
6. For each of the 10 images, **repeat steps 1–5**, generating **10 more TCP connections**.

Total = 11 TCP connections for 11 objects

Features of Non-Persistent HTTP

- Each connection handles **only one request–response pair**.
- **Multiple RTT delays** (Round-Trip Times) are required:
  - 1 RTT → TCP handshake
  - 1 RTT → HTTP request + response
    → **Total = 2 RTT per object (plus file transmission time)**
- Creates **significant overhead**:
  - More TCP connections
  - More server load
  - More delay for clients

Parallel TCP connections

- Modern browsers often open **5–10 TCP connections in parallel**.
- Speeds up downloading of multiple objects.
- User can also configure number of parallel connections.

---

2. Persistent Connections

Definition

A **single TCP connection** is kept open and reused for **multiple request–response pairs**.

How it works

- Server leaves the TCP connection **open** after sending a response.
- Client sends further requests on the **same** connection.
- Responses come back on the same channel.
- After some idle time, server **closes** the connection.

Advantages of Persistent Connections

1. Reduced Overhead

- No need to create a new TCP connection for each object.
- Saves CPU, time, and memory on both client and server.

2. Lower Latency

- Only the **first object** requires 2 RTTs.
- Subsequent objects need only **1 RTT** (request + response).

3. Pipelining (in persistent mode)

- Requests can be sent **back-to-back** without waiting for responses.
- Greatly improves performance for Web pages with many objects.

Comparison: Persistent vs Non-Persistent HTTP

Feature	**Non-Persistent**	**Persistent**
TCP connections	One per object	One for many objects
RTT penalty	2 RTT per object	1 RTT for all after first

Delay	High	Low
Server Load	High	Low
Supports pipelining	No	Yes
Efficiency	Poor	Excellent

RTT (Round-Trip Time) Calculation

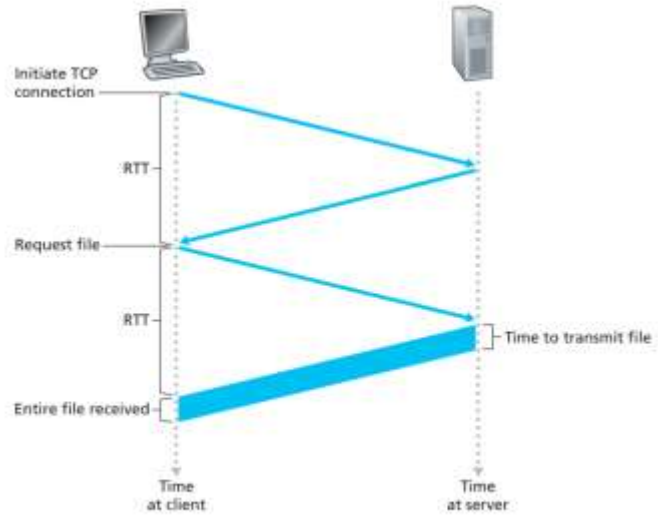

Figure 2.7 • Back-of-the-envelope calculation for the time needed to request and receive an HTML file

RTT = time for a small packet to travel from client → server → client.

For non-persistent HTTP:

- 1 RTT for TCP handshake
- 1 RTT for request + response
- Plus file transmission time

Total = 2 RTT + transmission time

Persistent connections eliminate repetitive 2-RTT costs.

HTTP Request Message

An **HTTP request message** is the message **sent by the client (browser)** to request a web object from a server.

Example Request Message

GET /somedir/page.html HTTP/1.1

Host: www.someschool.edu

Connection: close

User-agent: Mozilla/5.0

Accept-language: fr

Structure of an HTTP Request Message

An HTTP request message consists of:

1. Request Line

Contains three fields:

- **Method** (e.g., GET, POST, HEAD, PUT, DELETE)
- **URL** of the requested object
- **HTTP version**

Example:

GET /somedir/page.html HTTP/1.1

2. Header Lines

Additional information sent by the browser.

Examples:

- **Host:** Specifies hostname
- **Connection:** Indicates whether to keep or close the TCP connection
- **User-agent:** Browser type (Mozilla, Chrome, Firefox)
- **Accept-language:** Client's preferred language

3. Blank Line

Separates the header section from the body.

4. Entity Body (optional)

- Used mainly in **POST** requests.
- Contains user input (e.g., form submissions).

Common HTTP Request Methods

1. GET

- Most common method.
- Requests an object.
- Entity body is **empty**.

2. POST

- Used when submitting forms.
- Entity body contains user input.

Example:

username=abc&password=xyz

3. HEAD

- Same as GET but **no object is returned**.
- Used by developers for debugging.

4. PUT

- Uploads an object to the server (to a specified path).

5. DELETE

- Deletes an object on the server.

General Format of Request

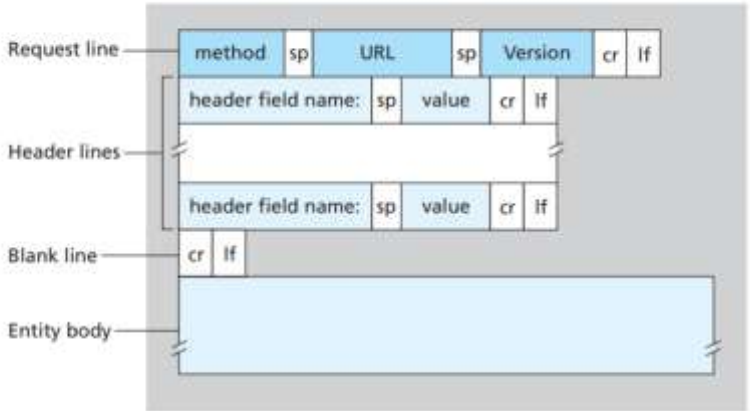

Figure 2.8 ♦ General format of an HTTP request message

- Request line
- Header lines
- Blank line

- Optional entity body

This standardized format ensures all clients and servers interpret requests correctly.

HTTP Response Message

An **HTTP response message** is sent by the server back to the client.

Example Response Message

HTTP/1.1 200 OK

Connection: close

Date: Tue, 09 Aug 2011 15:44:04 GMT

Server: Apache/2.2.3 (CentOS)

Last-Modified: Tue, 09 Aug 2011 15:11:03 GMT

Content-Length: 6821

Content-Type: text/html

(data data data data ...)

Structure of an HTTP Response Message

An HTTP response message consists of:

1. Status Line

Contains:

- **HTTP version**
- **Status code**
- **Status phrase**

Example:

HTTP/1.1 200 OK

2. Header Lines

Common header lines:

- **Connection:** Whether server will close or keep TCP connection
- **Date:** Time when response was created

- **Server:** Web server software
- **Last-Modified:** Time when requested object was last changed
- **Content-Length:** Number of bytes in body
- **Content-Type:** MIME type (e.g., text/html, image/jpeg)

3. Blank Line

4. Entity Body

Contains the actual object (HTML file, image, etc.).

Important HTTP Status Codes

1. 200 OK

- Request succeeded.
- Object is included in the response.

2. 301 Moved Permanently

- Requested object has moved.
- New URL given in Location: header.

3. 400 Bad Request

- Request cannot be understood by server.

4. 404 Not Found

- Requested document does not exist.

5. 505 HTTP Version Not Supported

- Server does not support the requested version.

Key Observations

1. HTTP messages are ASCII text

Readable by humans.

2. HTTP is a stateless protocol

Server does not store any client state between requests.

3. Header lines vary

Browsers, servers, and proxies add different headers based on configuration.

General Format of Response

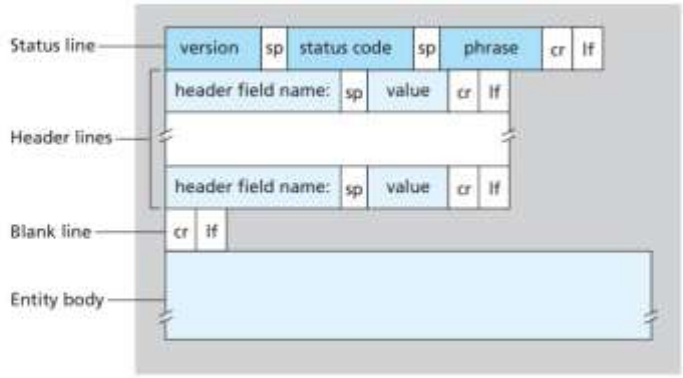

Figure 2.9 ♦ General format of an HTTP response message

- Status line
- Header lines
- Blank line
- Entity body

User–Server Interaction: Cookies

HTTP is a **stateless protocol**, which means the server does **not remember** anything about the user between requests.
However, many applications need to **track users**, **maintain sessions**, or **personalize content**. To achieve this, HTTP uses **cookies**.

---

1. What Are Cookies?

Cookies are small pieces of data used by web servers to:

- Identify users
- Maintain session state
- Provide personalized services
- Track user activity

Cookies are defined in **RFC 6265**.

---

2. Components of Cookie Technology

A complete cookie system has **four parts**:

1. Set-Cookie Header (in HTTP Response)

- Server sends a cookie to the browser using the **Set-cookie** header.
- Example:
- Set-cookie: 1678

2. Cookie Header (in HTTP Request)

- Browser sends the stored cookie back to the server with each request.
- Example:
- Cookie: 1678

3. Cookie File (Stored in Client Browser)

- Browser saves cookies on the user's device.
- Stores:
    - Hostname
    - Cookie values
    - Expiry date

4. Back-End Database (on Server)

- Server matches the cookie ID with user data stored in its database.

---

3. How Cookies Work (Step-by-step Example)

Scenario:

Susan visits Amazon using Internet Explorer **for the first time**.

Step 1: Susan sends normal HTTP request.

Step 2: Amazon server creates a unique ID, e.g., 1678.

Step 3: Server stores entry in database:

UserID = 1678

Step 4: Server sends response with:

Set-cookie: 1678

Step 5: Browser stores the cookie in the cookie file.

On Later Visits

When Susan visits Amazon again:

- Browser reads the cookie file.
- Sends:
- Cookie: 1678
- Amazon server retrieves Susan's stored information and acts accordingly:
    - Tracks which pages she visited
    - Shows personalized recommendations
    - Maintains shopping cart
    - Enables "one-click shopping"

---

4. What Cookies Allow Servers to Do

Cookies can be used to:

✓ Track user activity

- Pages visited
- Order of pages
- Time spent

✓ Maintain shopping carts

- Items added remain stored even without login

✓ Provide personalized services

- Recommendations
- Faster browsing

✓ Create user sessions on top of stateless HTTP

- Used in email services, e-commerce sites, banking portals

✓ Identify logged-in users

- Use cookies to maintain authentication state

---

5. Cookies and Privacy Issues

Cookies can be controversial because:

- They allow websites to track user behavior across multiple visits.
- Can collect personal information (with user login).
- Can be used by third-party advertisers to track browsing across websites.

Note: Cookies alone **do not store personal data**. Servers associate cookie ID with user data in their database.

---

6. Example of Cookie File Behavior

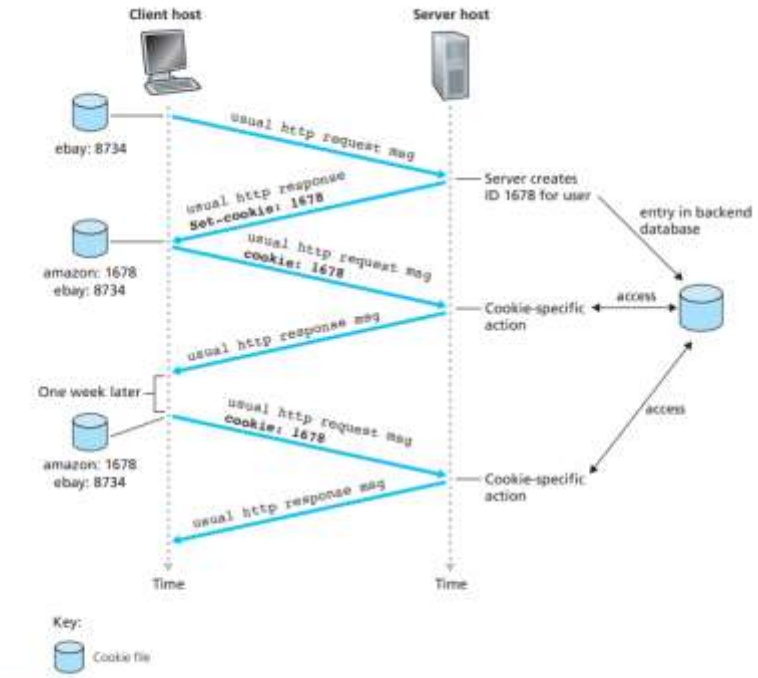

Figure 2.10 ♦ Keeping user state with cookies

The figure shows:

- Multiple websites storing cookies in the client's cookie file.
- Each site assigns its own ID (e.g., Amazon: 1678, eBay: 8734).
- On subsequent visits, browser sends back appropriate cookie.

This enables:

- Session continuation
- Personalized Amazon pages
- E-commerce features

---

7. Summary of How Cookies Enable State

Cookies allow a stateless HTTP protocol to support:

- **User identification**
- **Session management**
- **Shopping cart functionality**
- **Personalized content**
- **Authentication**

This is how sites enable features like:

- "Welcome back!"
- "Items in your cart"
- "Recommended for you"
- "One-click checkout"

Web Caching

1. What is a Web Cache?

A **Web cache**, also called a **proxy server**, is a network entity that:

- Receives HTTP requests from clients.
- Stores copies of recently requested web objects.
- Satisfies future requests for the same objects directly, without contacting the origin server.

Browsers can be configured so that **all HTTP requests go to the Web cache first**.

---

2. How a Web Cache Works (Steps)

Consider a user requesting:

http://www.someschool.edu/campus.gif

Step-by-step operation:

1. Client → Cache

The browser opens a TCP connection to the Web cache and sends the HTTP request.

2. Cache Lookup

The Web cache checks whether it already has the requested object stored **locally**.

3. If object is cached:

- Cache returns the object directly to the client.
- No contact with the origin server.
- Fast response.

4. If object is not cached:

- Cache opens its own TCP connection to the **origin server**.
- Sends its own HTTP request.
- Receives the object from the origin server.
- Stores it locally.
- Sends it back to the client.

Thus, a Web cache acts as:

- A **server** for clients
- A **client** for origin servers

---

3. Why Web Caching is Useful

Web caching provides **two major benefits**:

Benefit 1: Reduces Response Time

- Client–cache link is usually **high-speed LAN**, faster than Internet access link.
- If object is cached, the delay is extremely small (milliseconds).
- Even if not cached, caching reduces load on the origin server.

Benefit 2: Reduces Traffic on Access Links

- Cache reduces the number of requests sent across the institution's or ISP's Internet link.
- This reduces congestion on the access link.
- Institutions don't need to upgrade bandwidth frequently.
- Lower cost and improved performance.

---

4. Example: Web Cache Improving Performance

Network Setup

- Institutional LAN: **100 Mbps**
- Access link to Internet: **15 Mbps** (bottleneck)
- Average object size: **1 Mbit**
- 15 requests per second

Without caching:

- Traffic intensity on LAN:

$$0.15$$

- Traffic intensity on access link:

$$1.0 (\text{fully saturated = long delay})$$

Internet delay: ~2 seconds
Total response time: **minutes** (unacceptable)

Solution 1: Increase Access Link Bandwidth

Increase from **15 Mbps → 100 Mbps**:

- Traffic intensity becomes 0.15
- Delay becomes negligible
- Response time: roughly **2 seconds** (Internet delay)

BUT

- Very expensive
- Requires heavy infrastructure upgrade

Solution 2: Install a Web Cache (Better Solution)

Assume:

- Cache hit rate = **0.4** (40% of requests satisfied locally)

Effects:

- 40% requests satisfied instantly from LAN
- 60% requests go to Internet

Traffic intensity on access link becomes:

$$0.6$$

This creates small delay (~milliseconds).

Total response time:

$$0.4(0.01 \text{ sec}) + 0.6(2.01 \text{ sec}) \approx 1.2 \text{ sec}$$

Advantages

- Faster than upgrading bandwidth
- Cheaper and easier to deploy
- Greatly reduces Internet traffic for the institution

---

5. Additional Notes for Exams

Web cache is both:

- A **server** (when serving clients)
- A **client** (when fetching from origin servers)

Installed by:

- Large ISPs
- Universities
- Organizations
- Residential networks

Hit Rate

- Fraction of requests satisfied by the cache.
- Typically **0.2 – 0.7** in practice.

Improves Internet performance globally

- Reduces load on backbone networks.
- Reduces server workload.

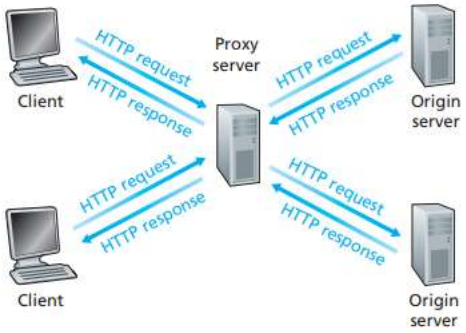

Figure 2.11 ♦ Clients requesting objects through a Web cache

- Client sends request to cache.
- Cache either:
    - Responds immediately (hit), or
    - Fetches from origin server (miss).
- Cache stores fetched objects for future use.

This cycle repeats for all client requests.

The Conditional GET

Web caching improves performance, but it introduces a problem:

The cached object may become stale.

The original object on the server may have changed since the cache stored its copy.

To solve this, HTTP provides a mechanism called the **Conditional GET**, which lets caches verify whether their stored object is still up to date.

What is a Conditional GET?

A request is a **Conditional GET request** if:

1. The HTTP method is GET

2. The request includes an If-Modified-Since header line

This lets the cache ask the server:

"Send the object **only if** it has been modified after this date."

Purpose of Conditional GET

- Avoids sending entire objects unnecessarily
- Saves bandwidth
- Reduces delay
- Ensures cached data stays fresh

How Conditional GET Works (Step-by-Step Example)

Step 1: Initial Request

A proxy cache requests:

GET /fruit/kiwi.gif HTTP/1.1

Host: www.exoticcuisine.com

Step 2: Server Response

Server sends the object with headers:

HTTP/1.1 200 OK

Date: Sat, 8 Oct 2011 15:39:29

Server: Apache/1.3.0

Last-Modified: Wed, 7 Sep 2011 09:23:24

Content-Type: image/gif

(data data data ...)

Cache stores:

- The object
- The **Last-Modified** timestamp

Step 3: A Week Later – Request Again

Browser requests the same image.
Cache uses conditional GET to check freshness:

GET /fruit/kiwi.gif HTTP/1.1

Host: www.exoticcuisine.com

If-Modified-Since: Wed, 7 Sep 2011 09:23:24

The value in **If-Modified-Since** is exactly the timestamp previously received.

Step 4: Server Response

Server checks if object has been modified since that date.

If **not modified**, server replies:

HTTP/1.1 304 Not Modified

Date: Sat, 15 Oct 2011 15:39:29

Server: Apache/1.3.0

(empty entity body)

Meaning of 304 Not Modified

- Object unchanged since given date
- No need to send object again
- Cache should use its stored copy

This avoids wasting time and bandwidth.

Key Features of Conditional GET

- **Ensures cached content is up-to-date**
- **Saves bandwidth — no need to resend same object**
- **Reduces response time for large objects**
- **Helps maintain HTTP caching efficiency**

Important Headers

Last-Modified

- Sent by server
- Indicates last time object was changed

If-Modified-Since

- Sent by client/cache
- Asks server to verify if object changed since this date

If unchanged → **304 Not Modified**
If changed → server sends updated object with **200 OK**

Why Conditional GET is Important

- Web caching alone might send stale data
- Conditional GET prevents outdated content
- Ensures consistency between client cache and server data
- Makes caching reliable and efficient

File Transfer: FTP

1. What is FTP?

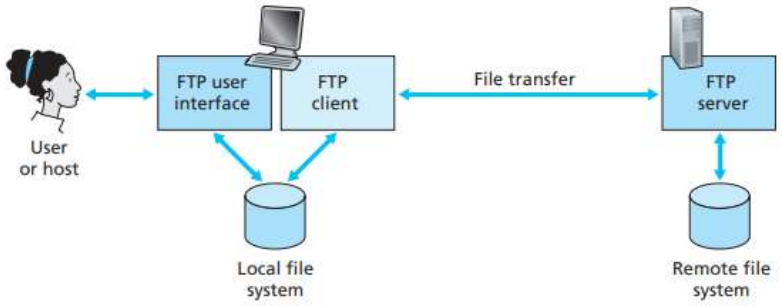

Figure 2.14 ♦ FTP moves files between local and remote file systems

- A network application-layer method used to **move stored items** between a **local system** and a **remote system**.
- Operates using **TCP**, ensuring reliability.
- A user sits at a local machine and communicates with a remote host that stores items.

---

2. Basic Working of an FTP Session

1. User enters a **user name** and **secret key** to access the remote system.
2. The **client side** of this method establishes a **TCP link** to the server side.
3. Once validated, a user may **obtain** items or **place** items on the remote system.
4. Items are moved between the **local store** and the **remote store**.

---

3. Components

(a) User or Host

- Person operating the system.

(b) FTP User Interface

- Menu/GUI/command area.

(c) FTP Client

- Runs on sender's system, sends all commands.

(d) FTP Server

- Runs on remote system, listens for incoming links and commands.

---

4. Control Link and Data Link

The most notable detail:

FTP uses *two* parallel TCP links:

(1) Control Link (port 21)

- Sends commands like login, change directory, etc.
- Stays active for the entire session.
- Only small text commands move here.

(2) Data Link (port 20)

- Carries the actual items.
- Opened **only when an item needs to move**, then closed.
- A fresh data link is created for every item transfer.

In-band vs. out-of-band

- **HTTP** mixes commands and data in one link → **in-band**.
- **This method keeps commands and data separate → out-of-band**.

---

5. State Maintenance

- The server must keep track of:
    - User ID
    - Current directory
    - Session details
- Thus, **stateful**.

- In contrast, **HTTP is stateless**.

---

6. Common Commands (Client → Server)

- **USER name** – sends user name.
- **PASS secret** – sends secret key.
- **LIST** – asks for names of items in the current remote location.
- **RETR name** – obtains an item from remote system.
- **STOR name** – places an item into remote system.

---

7. Common Replies (Server → Client)

Three-digit reply codes, such as:

- **331** – User name OK, secret needed.
- **125** – Data link open; moving items now.
- **425** – Data link cannot open.
- **452** – Cannot write item.

---

8. Advantages

- Reliable due to TCP.
- Separating command and item movement improves control and management.

---

9. Disadvantages

- Requires two links → more overhead.
- Stateful → server memory usage increases.
- Security concerns (plain login details in basic version).

Electronic Mail in the Internet – Study Notes (8–10 Marks)

1. Introduction to Electronic Mail

- Electronic mail (e-mail) is one of the oldest and most widely used Internet applications.
- It is an **asynchronous communication** method: the sender and receiver do not need to be online at the same time.

- Modern e-mail supports attachments, HTML content, embedded media, hyperlinks, etc.
- E-mail remains one of the most important applications on the Internet.

---

2. Main Components of Internet E-mail

Internet e-mail has **three major components**:

a) User Agents (UA)

- Software used by users to read, write, reply, forward, and organize e-mails.
- Examples: Outlook, Apple Mail.
- When the sender finishes composing a message, the UA sends it to the sender's mail server.

b) Mail Servers

- Mail servers store users' mailboxes.
- A **mailbox** is a storage location for incoming messages.
- Mail servers also maintain an **outgoing message queue** for messages waiting to be delivered.
- If a message cannot be delivered (receiver's server down), the server keeps retrying and later notifies the sender if it fails.

c) Simple Mail Transfer Protocol (SMTP)

- SMTP is the **primary application-layer protocol** used to transfer e-mail between mail servers.
- It uses **TCP** for reliable communication.
- It has two sides:
  - **SMTP client** → runs on the sender's mail server.
  - **SMTP server** → runs on the receiver's mail server.
- SMTP uses **persistent connections** if multiple messages are sent between the same servers.

---

3. How SMTP Works (Sending Email Example: Alice → Bob)

1. Alice composes a message using her user agent and sends it.

2. The message is placed in the **message queue** at Alice's mail server.
3. The **SMTP client** at Alice's server connects to Bob's mail server (using TCP port 25).
4. After initial SMTP handshake (greetings), Alice's server transfers the message.
5. Bob's mail server receives the message and stores it in **Bob's mailbox**.
6. Bob opens his user agent and reads the message whenever he wants.

---

4. Basic SMTP Commands

SMTP uses simple text (7-bit ASCII) commands, such as:

- **HELO** – Greet the server / introduce the client.
- **MAIL FROM:** – Specify the sender's e-mail address.
- **RCPT TO:** – Specify the recipient's e-mail address.
- **DATA** – Indicates that the message body follows.
- **QUIT** – End the session.

Messages end with a **single period (.) on a line by itself**.

---

5. SMTP Limitations

- SMTP handles **only 7-bit ASCII text**.
- Multimedia files, images, videos, etc., must be **encoded** (like Base64) before sending.
- Modern systems overcome this using MIME (Multipurpose Internet Mail Extensions), but basic SMTP still has the limitation.

Mail Message Formats (RFC 5322)

When an e-mail is sent from one user to another, the **message has two parts**:

1. Header lines

- Contain **peripheral information** (metadata).
- Each header is a **keyword followed by a colon and a value**.
- Headers and body are separated by a **blank line (CRLF)**.
- Common required headers:
    - **From:** sender's address

    - **To:** receiver's address
    - **Subject:** optional but commonly used
- Headers defined by **RFC 5322**.
- These headers are **different from SMTP commands** like MAIL FROM, RCPT TO.

Example mail header

From: alice@crepes.fr

To: bob@hamburger.edu

Subject: Searching for the meaning of life.

2. Message Body

- Plain ASCII text (originally).
- Comes **after a blank line**.

Comparison of SMTP and HTTP

SMTP and HTTP are both application-layer protocols that transfer files, but they have **fundamental differences**.

1. Pull vs Push

- **HTTP = Pull protocol**
    - Client requests data when it wants.
- **SMTP = Push protocol**
    - Sending mail server pushes the message to the receiving mail server.

2. Encoding Difference

- **SMTP requires 7-bit ASCII** for every message.
    - If message contains non-ASCII characters (e.g., accents, images), it must be **encoded** (Base64, MIME).
- **HTTP does not have this restriction**
    - Can transfer binary files directly.

3. Object Handling

- **HTTP** sends each object separately (HTML, images, CSS → separate responses).
- **SMTP** sends all message components (text + attachments via MIME) **in one message**.

Mail Access Protocols – POP3, IMAP, and Web-Based Email

When SMTP delivers Alice's mail to Bob's **mail server**, the message is stored in Bob's **mailbox**. Bob still needs a way to **access** it from his device.
This is done using **Mail Access Protocols**.

Mail access protocols help users **retrieve, read, and manage** their emails stored on a remote mail server.

The three major mail access methods are:

---

1. POP3 (Post Office Protocol – Version 3)

POP3 is a **simple**, **old**, and widely used mail access protocol defined in RFC 1939.

How POP3 works

POP3 operates in **three phases**:

(a) Authorization phase

- User connects to POP3 server on port **110**.
- User provides **username** and **password**.
- Example:
- user bob
- pass hungry

(b) Transaction phase

- User retrieves emails.
- User can delete emails or mark them for deletion.
- Commands used:
    - list – list messages
    - retr – retrieve message
    - dele – delete message
    - quit – end session

Example:

C: retr 1

S: (message content)

C: dele 1

(c) Update phase

- After quit, server deletes messages marked for deletion.

Two modes in POP3

1. Download-and-delete

- Messages removed from server after download.
- Problem: Not suitable for users who use **multiple devices** (PC, mobile, laptop).

2. Download-and-keep

- Messages stay on server.
- Allows access from multiple devices.
- But POP3 still does **not store folder information** or message states.

Advantages of POP3

- Simple and easy to implement.
- Fast because everything is downloaded locally.

Limitations of POP3

- Does **not** maintain state across sessions.
- No folder-based message organization on server.
- Not ideal for modern multi-device email access.

2. IMAP (Internet Mail Access Protocol)

IMAP is a more **advanced** and **feature-rich** mail access protocol than POP3, defined in RFC 3501.

Key features of IMAP

1. **Messages stay on the server**
   – Users access mail from many devices easily.
2. **Server-side folders**
   – Users can create folders, organize mails, search mails on the server.

3. **Stateful protocol**
   – Server keeps track of:
   - Read/unread status
   - Folder hierarchy
   - Which messages belong to which folders
4. **Partial download**
   – Useful for slow internet.
   – User can download only part of a large message (e.g., header first).

IMAP is ideal for

- Users who access email from **multiple devices**.
- Modern systems (Gmail, Yahoo Mail, Outlook Webmail often use IMAP).

Differences: POP3 vs IMAP

POP3	**IMAP**
Downloads emails to local device	Emails stored on server
Minimal features	Many features (folders, search)
Does not keep state	Keeps message state
Not ideal for multiple devices	Best for multiple devices
Simple	Complex

---

3. Web-Based Email (Gmail, Yahoo, Outlook Webmail)

Web-based email uses **HTTP** instead of POP3/IMAP for mail access.

How it works

- User logs into mail account through browser (Chrome, Edge, etc.).
- Browser communicates with the **mail server** using **HTTP/HTTPS**.
- When viewing inbox:
  – Mail sent from server → browser via HTTP
- When sending a message:
  – Mail sent from browser → sender's mail server using HTTP
  – Afterwards SMTP is used to deliver mail to other mail servers.

Key points

- User agent = **web browser**.
- Does **not** use POP3 or IMAP for reading email.
- SMTP still used for **sending** between mail servers.

Feature	SMTP	POP3	IMAP	HTTP (Webmail)
Purpose	Send mail between servers	Download mail	Manage mail on server	Access mail through browser
Direction	Push	Pull	Pull + manage	Pull via browser
Keeps state?	No	No	Yes	No
Multi-device friendly?	Not relevant	No (unless keep mode)	Yes	Yes

DNS – The Internet's Directory Service

Just like humans can be identified by many types of names—such as names, IDs, or license numbers—**Internet hosts also need identifiers**. For computers on the Internet, two types of identifiers are used:

1. **Hostnames**
2. **IP Addresses**

Both are important, but each serves a different purpose.

---

1. Hostnames

- These are **human-friendly names**, easy to read and remember.
  Example:
  - cnn.com
  - www.yahoo.com
  - gaia.cs.umass.edu
- Hostnames are **mnemonic**, meaning they are designed for people.
- They usually contain **words**, **dots**, and **letters**.

Limitations of Hostnames

- Hostnames **do not reveal the exact network location** of a device.
- They are **too long and variable** for routers to process efficiently.

- Example:
  www.eurecom.fr → ".fr" tells only that the host is in France, nothing more.

Because routers need fast and precise addressing, hostnames are not enough.

---

2. IP Addresses

- An **IP address** uniquely identifies each host on the Internet.
- It is a **32-bit number** (for IPv4), divided into four bytes.
  Example:
- 121.7.106.83
- Each part ranges from **0 to 255**.

IP addresses are hierarchical

- As you read from **left to right**, the address gives more detailed location information.
- This is similar to a **postal address**:
  - Country → State → City → Street → House number
- In IP addressing:
  - Network part → Sub-network → Host part

Routers use IP addresses (not hostnames) because they are **structured and easy to process**.

Why do we need DNS?

Humans prefer **hostnames** (easy to remember),
but computers and routers use **IP addresses** (precise for routing).

DNS solves this problem by acting as the **Internet's directory service**, mapping hostnames to IP addresses.

Example:

www.google.com → 142.250.195.46

DNS allows humans to use hostnames, while the network uses IP addresses internally.

Services Provided by DNS

DNS (Domain Name System) is the Internet's directory service. It translates **human-friendly hostnames** (like *www.google.com*) into **IP addresses** (like *142.250.183.4*), which are required for routing packets in the network.

DNS provides the following major services:

---

1. Hostname → IP Address Translation

This is the primary function of DNS.

Why needed?

- Humans prefer easy-to-remember names (e.g., *amazon.com*).
- Routers use numerical IP addresses.

How it works (process flow):

1. User runs an application (like a browser).
2. Browser extracts hostname from the URL.
3. Browser sends a DNS query to the DNS client.
4. DNS client sends the query to a DNS server.
5. DNS server replies with the IP address.
6. Browser uses this IP to establish a TCP connection to the web server.

This extra step introduces delay, but caching helps reduce it.

---

2. Host Aliasing

Many servers have complicated canonical hostnames.

Example:

Canonical hostname:
relay1.west-coast.enterprise.com

Aliases:

- **enterprise.com**
- **www.enterprise.com**

DNS allows mapping multiple alias names to one canonical name.

Purpose:

- Make names easier for users.
- Allow companies to change canonical names without affecting user access.

DNS returns:

- Canonical hostname
- IP address

---

3. Mail Server Aliasing

Email addresses also use DNS.

Example email: **bob@hotmail.com**

However, the actual mail server hostname may be long and complex.
DNS helps map the simple email domain (hotmail.com) to the correct mail server.

MX Record (Mail Exchanger):

- Special DNS record used to find the mail server for a domain.
- Allows email software to find the correct server automatically.

---

4. Load Distribution

DNS helps distribute client traffic across multiple replicated servers.

How?

A domain (e.g., *cnn.com*) might have several servers:

- Each with a different IP address.
- All mapped to the same hostname.

DNS replies with **a set of IP addresses**, often in rotating order (round robin).

Benefits:

- Distributes user load across multiple servers.
- Increases availability and performance.

Used in:

- Web servers
- Email servers
- Large content distribution networks like Akamai

---

5. Distributed, Hierarchical Database

DNS is implemented as a large distributed system consisting of:

- Root DNS servers
- TLD servers (.com, .org, .edu)
- Authoritative DNS servers

No single server stores the entire database; this increases reliability and scalability.

---

6. Application-Layer Protocol

- DNS is an **application-layer protocol**.
- Communicates between end systems using the client-server model.
- Uses **UDP on port 53** (fast, simple).

DNS performs:

- Querying
- Replying
- Caching

---

7. DNS Caching

Caching saves IP addresses from recent lookups to:

- Reduce DNS lookup delay
- Reduce overall network traffic

Browsers, operating systems, and DNS servers all store DNS cache entries.

Overview of How DNS Works

DNS (Domain Name System) is the **Internet's directory service**. Its job:
Translate hostnames (like www.google.com) into IP addresses (like 142.250.183.110).

How DNS Works (High Level)

When an application (like a browser) wants to access a website, it must know the site's IP address.
Steps:

1. The browser extracts the hostname (e.g., *www.someschool.edu*).
2. The computer calls the **DNS client** to resolve this hostname.
3. DNS sends a **query message** over UDP on port **53**.

4. After a delay, a **DNS reply** arrives containing the IP address.
5. This mapping is returned to the browser.

DNS appears simple to the user, but actually:
➡️ It is a *huge global system* of many DNS servers working together.

Why Not Use One Central DNS Server? (Problems)

A single centralized DNS server would be a bad idea. Problems include:

1. Single Point of Failure

If the one server crashes, the entire Internet's hostname lookup fails.

2. Traffic Volume

One server would need to handle **all DNS queries in the world**, causing overload.

3. Distant Centralized Database

Clients far away (like Australia to New York) would suffer high delays.

4. Maintenance Issues

One server cannot store records of every host on the Internet and update constantly.

➡️ Therefore, **DNS must be distributed**.

Distributed, Hierarchical DNS Database

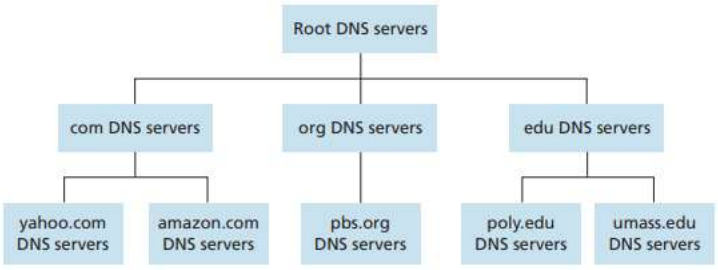

Figure 2.19 ♦ Portion of the hierarchy of DNS servers

DNS is implemented using **millions of servers**, structured in a **hierarchy**:

Three Main Levels of DNS Servers

1. **Root DNS Servers**
    - 13 logical root servers (A–M), each replicated globally.
    - Provide IP addresses of TLD servers.

2. **Top-Level Domain (TLD) DNS Servers**
    - Responsible for domains like **.com, .org, .net, .edu, country codes like .in or .uk**.
    - Example: Verisign manages **.com**.
3. **Authoritative DNS Servers**
    - Store actual domain-to-IP mappings.
    - Example: DNS server for **amazon.com** stores the IP of Amazon's web servers.

Local DNS Server

- Every ISP/university/company has one.
- Acts as *proxy* between client and global DNS hierarchy.
- Usually closest physically → reduces network delay.

DNS Lookup Example (From cis.poly.edu to gaia.cs.umass.edu)

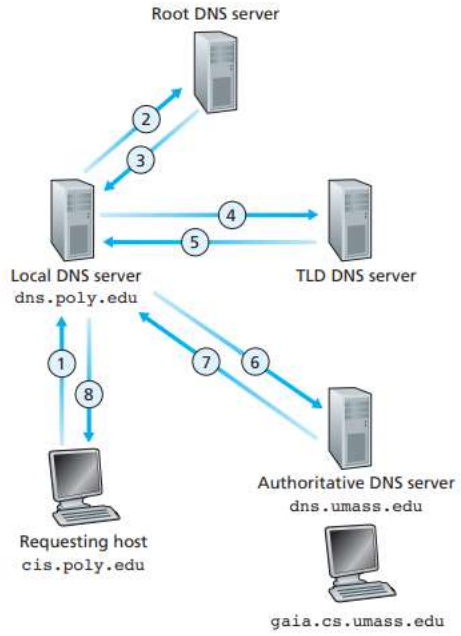

Figure 2.21 ♦ Interaction of the various DNS servers

Steps:

1. **Client (cis.poly.edu)** sends query to its **local DNS server**.
2. Local DNS server forwards query to a **root server**.
3. Root → sends IP of **TLD server (.edu)**.

4. Local DNS → sends query to **.edu TLD server**.
5. TLD → responds with the IP of **authoritative server for umass.edu**.
6. Local DNS → queries authoritative server.
7. Authoritative server returns IP of **gaia.cs.umass.edu**.
8. Local DNS gives IP address to client.

Total: 8 messages.

Recursive and Iterative Queries

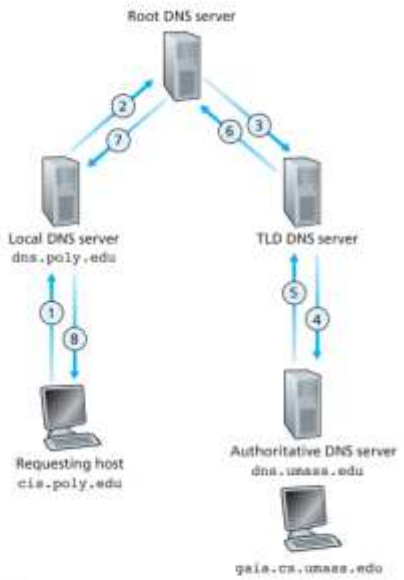

Figure 2.22 ♦ Recursive queries in DNS

Recursive Query

Client says:
➡️ "You find the answer and return it to me."
Local DNS does recursion on behalf of client.

Iterative Query

DNS server replies:
➡️ "I don't know, but try this server."
Example: root → TLD → authoritative.

Most real DNS lookups use:
➡️ Recursive from client → Local DNS
➡️ Iterative among DNS servers

DNS Caching

DNS servers store (cache) responses temporarily.

Benefits:

- Reduces **delay**
- Reduces **DNS traffic load**
- Makes DNS lookup faster for frequently visited domains

Example:

1. Host A asks for **cnn.com** → query travels the full hierarchy.
2. Host B asks again soon → local DNS returns cached IP instantly.

DNS cache entries expire (typically **2 days**) because IP addresses sometimes change.

DNS Records and Messages

1. DNS Resource Records (RRs)

DNS stores information in the form of **Resource Records (RRs)**.
Each RR is a **4-tuple**:

(Name, Value, Type, TTL)

- **Name** → The domain name.
- **Value** → Depends on RR type (IP address, hostname, etc.).
- **Type** → Indicates the kind of record (A, NS, CNAME, MX).
- **TTL** → Time-to-live (how long the record stays in cache).

2. Types of DNS Records

(a) Type A (Address Record)

- Maps a **hostname → IP address**.
- Example:
  (relay1.bar.foo.com, 145.37.93.126, A)

(b) Type NS (Name Server Record)

- Indicates the **authoritative DNS server** for the domain.
- Used to route queries further along the hierarchy.

- Example:
  (foo.com, dns.foo.com, NS)

(c) Type CNAME (Canonical Name Record)

- Maps an **alias hostname → canonical hostname**.
- Helps avoid storing IPs for multiple aliases.
- Example:
  (foo.com, relay1.bar.foo.com, CNAME)

(d) Type MX (Mail Exchange Record)

- Maps a domain → **mail server hostname**.
- Allows multiple mail servers / aliases.
- Example:
  (foo.com, mail.bar.foo.com, MX)

---

3. How RRs Are Stored in DNS Servers

- Authoritative DNS servers store **Type A** records for hosts in their domain.
- If not authoritative, a server holds **NS records** pointing to the authoritative server.
- Example:
  The .edu TLD server contains:
  (umass.edu, dns.umass.edu, NS)
  (dns.umass.edu, 128.119.40.111, A)

---

4. DNS Messages

DNS uses only **two message types**:

1. **Query message**
2. **Reply message**

Both use the same format.

DNS Message Format

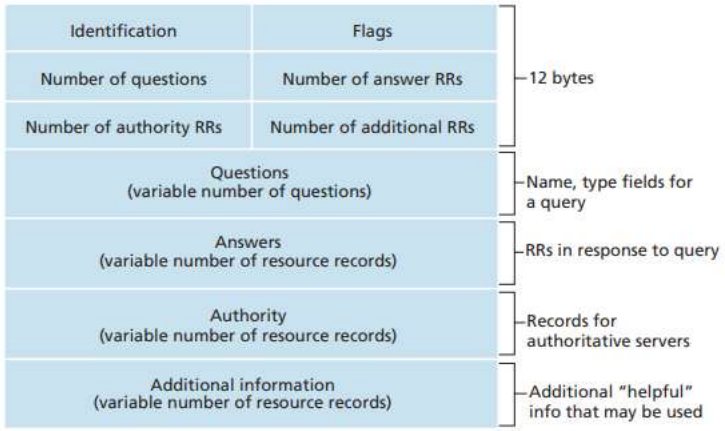

Figure 2.23 ♦ DNS message format

(a) Header Section (12 bytes)

Contains:

- **Identification** → matches query & reply.
- **Flags**:
  - Query (0) / Reply (1)
  - Authoritative (AA)
  - Recursion Desired (RD)
  - Recursion Available (RA)
- **Counts**:
  - Number of Questions
  - Number of Answer RRs
  - Number of Authority RRs
  - Number of Additional RRs

(b) Question Section

Contains:

- **Name** → the domain queried
- **Type** → A, NS, CNAME, MX, etc.

(c) Answer Section

Contains RRs that provide the **actual answer** for the query.
Example: host's IP addresses (Type A) or mail server info (MX).

(d) Authority Section

Contains RRs for **authoritative name servers**.

(e) Additional Information Section

Gives extra helpful RRs.
Example: When returning an MX record, this section may include the **Type A record** for the mail server.

---

5. Using nslookup

- nslookup is a tool for querying DNS servers manually.
- You can query root, TLD, or authoritative servers directly.
- Displays actual DNS replies in a readable format.

---

6. Inserting Records into DNS

- Domain names are registered through a **registrar** (accredited by ICANN).
- Registrar adds:
    - NS records
    - A records for authoritative servers
- Example entries:
- (networkutopia.com, dns1.networkutopia.com, NS)
- (dns1.networkutopia.com, 212.212.212.1, A)

---

7. DNS Security Issues

DNS can be attacked through:

(a) DDoS Attacks

- Flood DNS servers (especially root and TLD servers).
- However, DNS is highly resilient due to replication.

(b) Man-in-the-Middle Attacks

- Attackers intercept DNS replies and send forged responses.

(c) DNS Poisoning / Spoofing

- Attackers inject false records to redirect users to fake websites.

(d) DNS-Based Amplification Attacks

- Small queries trigger large responses → used to overload victims.

DNS survives attacks due to:

- Distributed hierarchy
- Replication
- Caching
- Security configurations